UNIVERSITY OF ARKANSAS

CSCE 4133/5133 Algorithms

Homework Assignment #4

Submission Deadline: Mid-Night (11:59PM) April 13, 2026

Instructions

- **Submissions:** Students can use either Google Docs or LaTeX for writing the report. The report template is provided in the Blackboard (Material/Report-template.zip). Your submission should be a zip file containing your report and the source code, including your implementation. Name the zip file as `lastname_studentID.zip`. Submissions should be made via BlackBoard.
- **Policy:** Students must start working on this assignment independently and review the late-day policy.
- If you have any questions please contact us via Teams or email: kimtran@uark.edu and maalexra@uark.edu.

I. Overview

In this assignment, we are **Required** to implement sorting algorithms, the minimum spanning tree algorithms, the shortest path algorithm, and Balancing Tree algorithm i.e., **Heap sort, MergeSort, Topological sort, Prim's Algorithm, Kruskal's Algorithm, Dijkstra Algorithm and Red-Black Tree**. This is a long HW, so start early. There are some hints that may help you succeed in this homework:

- You should start your homework early.
- You should read the descriptions of the homework carefully before starting work on the homework.
- Before implementing the required functions, you should first try compiling the source code. Basically, the source code with an empty implementation can be compiled successfully. This step will help you to understand the procedure of each problem.
- If you forgot these algorithms, you are encouraged to revise the lecture slides.

The following files are the only one you need to code in:

File or Folder	**Purpose**
src/sort/msort.py	Implement the Merge Sort algorithm
src/sort/hsort.py	Implement the Heap Sort algorithm
src/algorithms/topologicalsort.py	Implement the Topological sort
src/algorithms/prim.py	Implement Prim's Algorithm
src/algorithms/kruskal.py	Implement Kruskal's Algorithm
src/algorithms/dijkstra.py	Implement Dijkstra's Algorithm
src/algorithms/rbtree.py	Implement Red-Black Tree

II. Implementation

II..1 Heap Sort and Merge Sort

Merge Sort - you are given a method in scr/sort/msort.py as follows:

```
T = TypeVar("T")
def merge(array: List[T], l: int, m: int, r: int) -> None:
    # YOUR CODE HERE
    pass
def msort(array: List[T], l: int, r: int) -> None:
    # YOUR CODE HERE
    pass
```

Heap Sort - you are given a method in scr/sort/hsort.py as follows:

```
# Function to perform heap sort
def heap_sort(arr: List[int]) -> None:
    # YOUR CODE HERE
    pass
# Helper function to maintain the max-heap property
def heapify(arr: List[int], n: int, i: int) -> None:
    # YOUR CODE HERE
    pass
```

Please review the lecture slide and implement it. In this homework, you must implement these algorithms as optimal as possible. The expected complexity of these algorithms is O(n logn) where n the size of an array.

II..2 Graph and Disjoint Set Usages

Before you start implement algorithms, you first investigate the usages of Graph and Disjoint Set. This class has beealready beenplemented in the src/graph.py and src/data_strutures/disjoint_set.py.

Class	Method	Meaning
Graph	Graph	Initialize a graph with n vertices
	insert_edge	Insert an undirected (or directed) edge into the graph
	is_visited	Return true if the current vertex was already visited by the searching algorithm
	set_visited	Mark a vertex as visited vertex
	trace	Return the previous vertex of the given vertex in the searching order (return -1 if the vertex is the starting vertex in the searching order)
	set_trace	Set the previous vertex of the current vertex in the searching order
	search	Perform the search algorithm to find a path from start to destination
	export_edges	Export a list of edges
DisjoinSet	DisjointSet	Create a disjoint set
	is_on_same_set	Check two vertices is on the same or not
	join	Merge sets of two vertices into the same set

II..3 Topological Sort - Khan's Algorithm

You are given a method in src/algorithms/topologicalsort.py as follows:

```
def topological_sort(graph: Graph) -> List[int]:
```

```
    # Get the number of vertices in the graph
    n = graph.get_n()
    # Initialize in-degree array (number of incoming edges for each vertex)
    in_degree = [0] * n
    # YOUR CODE HERE
    return []
```

Given a Graph G, you have to implement Topological sort and return a sorted order of vertices in a directed acyclic graph.You need to use Khan's Algorithm. The implementation should ensure correctness, efficiency, and handle potential cycles in the graph.

II..4 Prim's Algorithm

You are given a method in src/algorithms/prim.py as follows:

```
def construct_mst_prim(G: Graph) -> List[Edge]:
    edges = G.export_edges()  # Graph's edges (may include both directions)
    n = G.get_n()
    mst: List[Edge] = []
    heap: List[tuple[int, int, int, Edge]] = []
    # you can use heap to optimize the minimum searching
    # YOUR CODE HERE
    return mst
```

Given a Graph G, you have to implement Prim's Algorithm and return a list of edges as the minimum spanning tree of G. We recommend using heap to optimize your minimum searching algorithm during performing Prim's algorithm.

II..5 Kruskal's Algorithm

You are given a method in src/algorithms/kruskal.py as follows:

```
def construct_mst_kruskal(G: Graph) -> List[Edge]:
    edges = G.export_edges()  # Graph's edges
    # DisjointSet djs(G.n);
    # Use Disjoint Set to check whether two vertices are on the same set
    # Usage: Check djs.is_on_same_set(u, v); Check is u and v is on the same set or not
    #               djs.join(u, v); Join sets of u and v into the same set
    mst: List[Edge] = []
    djs = DisjointSet(G.get_n())
    # YOUR CODE HERE
    return mst
```

Given a Graph G, you have to implement Kruskal's Algorithm and return a list of edges as the minimum spanning tree of G. We provide you a Disjoint Set implementation so that you can use this data structure to check whether two vertices are connected or not. Please make sure to implement **MERGE SORT** in this Algorithm.

II..6 Dijkstra's Algorithm

You are given a method in src/algorithms/dijkstra.py as follows:

```python
def search_shortest_path(G: Graph, start: int, destination: int) -> List[int]:
    G.reset()
    n = G.get_n()
    G.dist[start] = 0
    heap: List[tuple] = [(0, start)] # use heap to optimize the minimum searching
    path: List[int] = []
    # YOUR CODE HERE
    return path
```

Given a Graph G, you have to implement Dijkstra's Algorithm and return a list of vertices that is the shortest path from start to destination on graph G. We recommend using heap to optimize your minimum searching algorithm during performing Dijkstra's algorithm.

II..7 Red-Black Tree Algorithm

II..7.1 Objective

The goal of this assignment is to implement the balancing logic for a **Red-Black Tree (RBT)**. While search operations in an RBT are identical to a standard Binary Search Tree, insertions require specific rotations and recoloring to maintain a logarithmic height $O(\log n)$.

II..7.2 Properties to Maintain

Your implementation must ensure that after every insertion, the following properties remain true:

1. Every node is either **RED** or **BLACK**.
2. The root of the tree is always **BLACK**.
3. New leaves (NIL nodes) are considered **BLACK**.
4. If a node is **RED**, then both its children must be **BLACK** (No two Red nodes can be adjacent).
5. For each node, all paths from the node to descendant leaves contain the same number of black nodes.

II..7.3 Tasks

You are given 5 methods in src/algorithms/rbtree.cpp as follows:

```python
class RedBlackTree:
    def __init__(self):
        self.root: Optional[RBNode] = None

    def rotate_left(self, root: Optional[RBNode], ptr: Optional[RBNode]) -> None:
        # YOUR CODE HERE
        pass
```

```python

    def rotate_right(self, root: Optional[RBNode], ptr: Optional[RBNode]) -> None:
        # YOUR CODE HERE
        pass

    def fix_violation(self, root: Optional[RBNode], ptr: Optional[RBNode]) -> None:
        # YOUR CODE HERE
        pass

    def insert(self, data: int, x: int, y: int) -> None:
        # YOUR CODE HERE
        pass

    def search(self, data: int) -> Optional[RBNode]:
        temp = self.root
        while temp is not None:
            if data == temp.data:
                return temp
            if data < temp.data:
                temp = temp.left
            else:
                temp = temp.right
        return None
```

II..7.4 Rotation Logic

Implement `rotateLeft` and `rotateRight`. These are the fundamental operations used to rebalance the tree.

- Remember to update the `parent` pointers for the moved subtrees.
- Ensure the `root` pointer is updated if the rotation occurs at the top of the tree.

II..7.5 Standard BST Insertion

In the `insert` function:

- Implement a standard BST insertion based on the `data` value.
- New nodes must initially be colored **RED**.
- Once the node is placed, call `fixViolation` to restore RBT properties.

II..7.6 Violation Fixing

In `fixViolation`, handle the three primary cases of Red-Red violations:

- **Case 1:** Uncle is RED → Recolor parent, uncle, and grandparent.
- **Case 2:** Uncle is BLACK (Triangle) → Rotate parent to form a line.

- **Case 3:** Uncle is BLACK (Line) → Rotate grandparent and recolor.

III. Compilation and Testing

In this homework, we use Python3 compile the source code.

Before you implement any method, the output should look as follow:

```
Perform unit test on heap sort
Original array: 7 49 23 8 30 22 44 28 23 9 40 15 42 42 37 3 27 29 40 12
Sorted array: 7 49 23 8 30 22 44 28 23 9 40 15 42 42 37 3 27 29 40 12
Your sorting implementation is incorrect
```

If you implement all methods correctly, the unit test function should return "true" and you should receive an output as follows:

```
Perform unit test on heap sort
Original array: 7 49 23 8 30 22 44 28 23 9 40 15 42 42 37 3 27 29 40 12
Sorted array: 3 7 8 9 12 15 22 23 23 27 28 29 30 37 40 40 42 42 44 49
Your sorting implementation is correct
Perform unit test on Merge sort
Original array: 3 19 9 7 10 33 49 28 16 35 47 26 12 17 10 33 29 49 29 21
Sorted array: 3 7 9 10 10 12 16 17 19 21 26 28 29 29 33 33 35 47 49 49
Your sorting implementation is correct
Perform unit test on Topological sort
Topological Sort: 4 5 0 2 3 1
Topological Sort implementation is correct.
Perform unit test of the Prim's algorithm
Total Cost of Minimum Spanning Tree: 16
Total Cost of Minimum Spanning Tree: 9315
You have to use OpenCV to visualize your map
Perform unit test of the Kruskal's algorithm
Total Cost of Minimum Spanning Tree: 16
Total Cost of Minimum Spanning Tree: 9315
You have to use OpenCV to visualize your map
Perform unit test of the Dijkstra algorithm
Path from 0 to 5: 0 -> 1 -> 2 -> 4 -> 5
Path from SAEF to GRAD: SAEF -> BUCH -> LCAF -> GLAD2 -> PDTF -> SINF -> LMK111 -> WAHR ->
    SDPG2 -> ARKU2 -> POST -> FPAC -> WATR -> ARSAGAS -> CORD1 -> PTSC -> MEMH2 -> ARMY ->
    GRAD
You have to use OpenCV to visualize your map road
Perform unit test of the Red-Black Tree
==========================================================
--- STARTING INTEGRATED ALGORITHM STACK DEMO ---
==========================================================
[Step 1] RB-Tree: Coordinates stored for Buildings 0, 1, and 2.
 -> Found Building 0 at (100, 100)
 -> Found Building 1 at (200, 300)
```

```
 -> Found Building 2 at (400, 150)
[Step 2] Graph: Edges created with Euclidean weights:
 -> Edge (0,1) weight: 223
 -> Edge (1,2) weight: 250
 -> Edge (0,2) weight: 304
[Step 3] Kruskal: Constructing Minimum Spanning Tree...
 -> Added Edge (0,1) w:223
 -> Added Edge (1,2) w:250
Total MST Cost: 473
[Step 4] Dijkstra: Calculating shortest path from 0 to 2...
 -> Shortest Path: 0 -> 2
[Step 5] HeapSort: Sorting traffic congestion scores...
 -> Original: 85 12 44 98 30
 -> Sorted:   12 30 44 85 98
============================================================
--- INTEGRATED DEMO COMPLETE ---
============================================================
```

IV. Submission

Write a report detailing the key ideas for implementing each required function and include the implemented functions themselves. **Explain the run time complexity for each algorithm**. You must also include a screenshot of the final output (from either command prompt or terminal) after running the program.

Write your full name, email address and student ID in the report. Your submission should be a zip file of your report and the source code, including your implementation. Name the zip file as 'lastname_studentID.zip'. Submission via BlackBoard.